the LLM commercial service. At the time of writing, this feature wasn't an option for open source LLM deployments.

Next, we'll look at the bottom of the code in `first_function.py`, as shown in listing 5.2. Here are just two examples of calls made to an LLM using the request previously specified in listing 5.1. Here, each request shows the generated output from running the example.

Listing 5.2 `first_function.py` (exercising the API)

```
user = "Can you please recommend me a time travel movie?"
response = ask_chatgpt(user)     #1
print(response)

###Output
Function(arguments='{"topic":"time travel movie"}',
                      name='recommend')     #2

user = "Can you please recommend me a good time travel movie?"
response = ask_chatgpt(user)     #3
print(response)

###Output
Function(arguments='{"topic":"time travel movie",
                     "rating":"good"}',
 name='recommend')     #4
```

#1 Previously defined function
#2 Returned in the name of the function to call and the extracted input parameters
#3 Previously defined function
#4 Returned in the name of the function to call and the extracted input parameters

Run the `first_function.py` Python script in VS Code using the debugger (F5) or the terminal to see the same results. Here, the LLM parses the input request to match any registered tools. In this case, the tool is the single function definition, that is, the recommended function. The LLM extracts the input parameters from this function and parses those from the request. Then, it replies with the named function and designated input parameters.

> **NOTE** The actual function isn't being called. The LLM only returns the suggested function and the relevant input parameters. The name and parameters must be extracted and passed into a function matching the signature to act on the function. We'll look at an example of this in the next section.